Wave-image generation in Python

A tooling survey for parametric, reproducible ocean-stimulus synthesis

HumanNatureAttunement project
Antoine Bellemare & Mar Estarellas

April 30, 2026

Abstract

The HNA project couples brain / heart / muscle rhythms with the visual envelope of natural sea waves. The complementary need is to *generate* sea-wave clips with precise, dialed-in oscillation properties so that entrainment can be probed parametrically: "does a 0.25 Hz wave drive respiration more than a 0.10 Hz wave?" This report surveys the techniques available in 2026 to synthesise realistic, controllable sea-wave images and video from Python; recommends a 3-tier pipeline that spans the realism / cost trade-off; and gives the closed-form wind-speed-to-wave-period relationship that lets us hit a chosen neurophysiological target band (0.1–0.5 Hz) directly.

This document compiles with `tectonic report/wave_synthesis.tex`. The figures are produced by `scripts/wave_synthesis_figures.py`; the small Tessendorf FFT ocean snapshots inside it are themselves a working demo of the recommended Tier 1 pipeline.

Contents

1 Why a synthesis module

The analysis-side module `HNA.modules.video` reduces a sea-wave *recording* to a battery of 1-D signals (luminance envelope, optical-flow magnitude, modal coefficients, per-pixel temporal spectra, windowed complexity). This survey is the symmetric question: how do we *produce* sea-wave clips for which the wave period, foam content, fetch, depth, viewing angle and lighting are all known and controllable ahead of recording?

Why this matters. Naturalistic stimuli capture richer neurophysiological responses than artificial gratings, but they sacrifice parametric control. Synthesis closes the gap: a procedurally generated ocean clip is naturalistic *and* fully parameterised. With an explicit dominant frequency f_p we can:

- map dose-response curves of EEG envelope coupling vs. f_p within the autonomic / respiration band;
- match audio and visual envelopes by construction (test cross-modal entrainment without confounds);
- run condition replicates from a fixed seed for reproducibility.

Constraints. Whatever pipeline we adopt must (i) accept explicit numerical inputs (wind, fetch, depth, choppiness, foam, sun, camera, seed) rather than text prompts, (ii) run on Windows + CUDA with permissive licensing, and (iii) deliver MP4 at 30 fps for ∼60–120 s. These constraints rule out most generative-AI pipelines (Sec. 2.4) and favour spectrum-based procedural synthesis with optional photoreal rendering.

2 Families of techniques

2.1 Procedural mathematical surfaces

The dominant family in computer graphics for open-water animation since the 1980s. A heightfield $\eta(x, y, t)$ is computed analytically and rendered with shader code; no fluid PDE is solved.

Sum of sines and Gerstner waves. The simplest sum,

$$\eta(x, y, t) = \sum_{i=1}^{M} a_i \sin(\mathbf{k}_i \cdot \mathbf{x} - \omega_i t + \phi_i), \tag{1}$$

is fast and trivially controllable but flat-crested and unconvincing. Fournier & Reeves (1986) introduce trochoidal (Gerstner) waves, which add a horizontal displacement so crests sharpen: $\mathbf{x} \to \mathbf{x} - (\mathbf{k}_i/k_i) Q_i a_i \sin(\dots)$. Finch (2004) (*GPU Gems* ch. 1) gives the standard recipe. Dispersion $\omega = \sqrt{gk \tanh(kh)}$ couples wavenumber to depth.

Spectrum-based / Tessendorf FFT ocean. The state of the art for offline open-water (Pixar, Weta, EA, Unreal) since Tessendorf (2001). A 2-D wavenumber grid is sampled from a sea-surface spectrum – typically Phillips, Pierson–Moskowitz (Pierson & Moskowitz, 1964), or JONSWAP (Hasselmann *et al.*, 1973) – multiplied by time-evolved phasors $\exp(i\omega(\mathbf{k})t)$, and inverse-FFTed to a tileable heightfield (Fig. 1). Choppy waves are added by horizontally displacing the gradient field; foam is detected as the zero-set of the displacement Jacobian. The pipeline is (Mastin *et al.*, 1987)-style and amounts to ∼200 lines of NumPy/CuPy (see Fig. 2, where the snapshots are produced by `scripts/wave_synthesis_figures.py`).

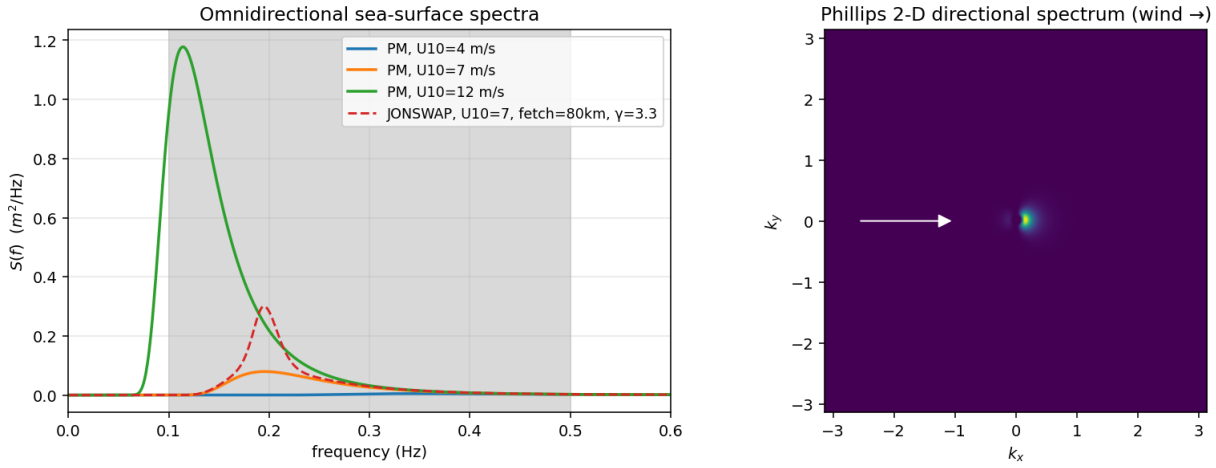

Figure 1: Left: omnidirectional Pierson–Moskowitz frequency spectra at three wind speeds, plus a JONSWAP variant for fetch-limited seas. The shaded band is the target neurophysiological range (0.1–0.5 Hz). Right: the corresponding 2-D directional Phillips spectrum used inside Tessendorf (2001)'s FFT ocean.

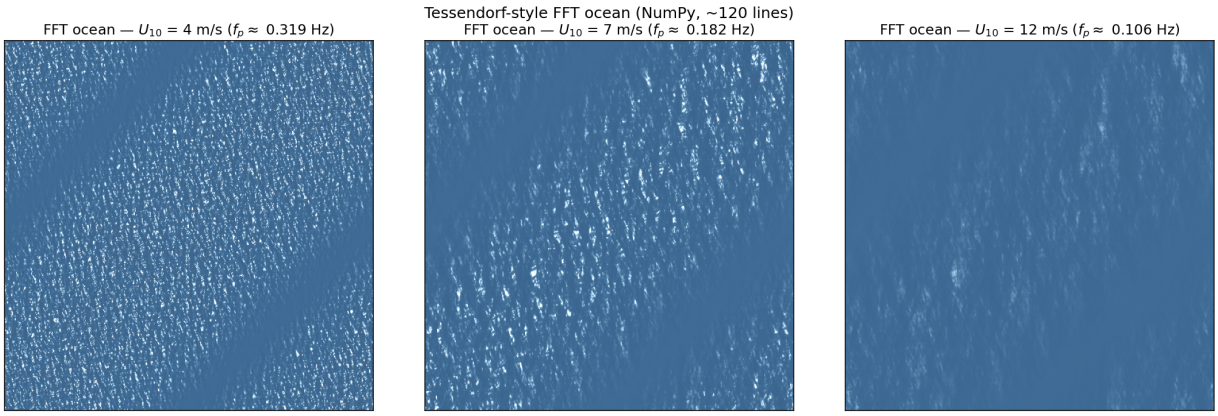

Figure 2: Tessendorf-style FFT ocean rendered with ∼120 lines of NumPy + cheap Lambertian shading. Three wind speeds, same seed; peak period varies from ∼3 s (U_{10}=4 m/s) to ∼9 s (U_{10}=12 m/s). With proper sky / specular shading and a foam layer this approach is the workhorse of the visual-effects industry.

Knobs the user controls. For Tessendorf-FFT: U_{10} (wind speed at 10 m), wind direction, fetch (JONSWAP), gravity g, depth h, patch size L, grid resolution N, choppiness λ, foam threshold, RNG seed.

Python implementations.

- **NumPy / CuPy from scratch.** ∼200 lines. Deterministic; easy to validate spectrally; runs in seconds on a CUDA GPU.
- **Taichi-lang** (Hu *et al.*, 2019) ships an FFT ocean example; Python-first, JIT-compiled to CUDA; reproducible via `ti.init(random_seed=...)`.
- **Mitsuba 3** (Jakob *et al.*, 2022) can render the heightfield (or its displacement map) with path-traced water; pure Python, BSD-3, CUDA via `LLVM/OptiX`.

2.2 Physics-based fluid simulation

Solve Navier–Stokes (or shallow-water, or SPH) on a grid and render the result. *For open-water visuals this is overkill* – the spectrum-based methods produce indistinguishable images at a tiny fraction of the cost. Worth knowing about if shore-break / splash close-ups matter later.

- **Eulerian Navier-Stokes:** *Stable Fluids* (Stam, 1999); `PhiFlow` (Holm *et al.*, 2024) (Apache, JAX/PyTorch); `jax-cfd` (Apache).
- **SPH:** `PySPH` (BSD, slow); `Genesis` (Genesis Team, 2024) (Apache, GPU, 2024+).
- **Position-based fluids:** Macklin & Müller (2013); bindings via `NVIDIA Flex` or `Taichi MPM`.
- **Lattice Boltzmann:** `lettucecfd/lettuce` (PyTorch, MIT).

Cost. Anywhere from seconds (2-D shallow-water) to minutes per frame (3-D NS with free surface). Realism depends almost entirely on the rendering layer, not the solver.

2.3 GPU shader synthesis from Python

Write a fragment shader (typically a raymarched water surface) and run it off-screen, dumping frames to MP4. Real-time even at 1080p.

- **moderngl** (MIT) – the canonical OpenGL bindings; load a GLSL shader, render to an off-screen FBO, capture frames via `imageio-ffmpeg`.
- **wgpu-py** (BSD) – modern WebGPU bindings; future-proof.
- **Taichi** – Python-first, JIT-to-CUDA, also runs on Apple Metal; particularly clean for a Tessendorf FFT ocean.

What to port. Inigo Quilez's *Seascape* Shadertoy (MIT) is the standard reference shader; it raymarches a heightfield built from nested noise + Gerstner waves, then shades with sky and Schlick Fresnel. Horvath (2015) extends with empirical directional spectra. All parameters are uniforms and trivially settable from Python.

2.4 AI / generative video synthesis

The newest family and the most tempting for naturalistic stimuli, but the worst fit for our use case for one structural reason: *text- and image-conditioned diffusion models do not let you specify a dominant oscillation frequency.*

- **Image:** Stable Diffusion + ControlNet via `diffusers` (Apache); produces stunning seascapes from a prompt but each frame is independent.
- **Video diffusion:** Stable Video Diffusion (Stability, *non-commercial* license – caveat for research outputs); AnimateDiff (Apache); CogVideoX (CogVideo Team, 2024) (Apache, 2024); Open-Sora (Apache, 2024); HunyuanVideo, Lumina-Video.
- **Cinemagraphs.** Holynski *et al.* (2021) animate water in a *single* still photo by warping along a learned (or user-painted) Eulerian motion field; the result loops smoothly but the temporal frequency emerges from the model and the painted field rather than from a numerical knob.
- **Diffusion-based fluids.** A small but growing literature (e.g. DiffusionPDE) – not a production tool yet.

Verdict for entrainment work. Use these only as a stylistic finish on top of a procedurally generated clip whose dominant frequency has already been validated spectrally; never as the source of f_p.

2.5 Hybrid & specialised

- **Procedural geometry → photoreal renderer.** Generate FFT/Gerstner heightfield in Python, feed it to (Jakob *et al.*, 2022) (BSD-3, Python-first) or **Blender Cycles** via `bpy` (GPL) for path-traced output. The *highest-realism reproducible* path.
- **Eulerian video magnification, in reverse.** (Wu *et al.*, 2012; Wadhwa *et al.*, 2013) amplify tiny oscillations; the same decomposition can *inject* a chosen frequency into an existing seascape video by modulating a phase-decomposed steerable pyramid. Niche but useful when you want to imprint, e.g., 0.2 Hz on real footage.
- **Houdini Python.** Industry standard FX, but commercial license; HQueue scripting works but heavyweight.
- **Blender "Ocean" modifier.** Implements Mastin *et al.* (1987) + Tessendorf choppiness + foam. Fully scriptable from `bpy`: knobs are `spatial_size`, `resolution`, `wind_velocity`, `depth`, `choppiness`, `foam_coverage`, `random_seed`, animated over `time`. Renders with Cycles (path-traced, CUDA/OptiX) or Eevee (real-time-ish). *The single best off-the-shelf option for this project.*

2.6 Rendering hosts callable from Python

Host	Python	Notes
Blender Cycles / Eevee	`bpy` (PyPI, GPL)	Ocean modifier built-in, HDRI sky, foam shaders, headless via `blender -background -python`.
Mitsuba 3	`mitsuba` (PyPI, BSD-3)	physically-based, differentiable, CUDA via `LLVM`; bring your own geometry.
PyVista	`pyvista` (MIT)	quick procedural meshes, not photoreal.
Three.js	`pythreejs`	awkward but possible with Playwright screenshots.

3 Frequency-targeting cheat sheet

For a fully developed sea, the Pierson–Moskowitz peak frequency is (Pierson & Moskowitz, 1964)

$$f_p \approx \frac{0.13\,g}{U_{10}} \quad \Longleftrightarrow \quad U_{10} \approx \frac{0.13\,g}{f_p}. \tag{2}$$

This single relation lets us go from a desired neurophysiological target band straight to an actionable wind-speed input (Fig. 3).

4 Recommended pipeline tiers

4.1 Tier 1 – NumPy/CuPy FFT ocean (fastest)

A ∼200-line implementation of Tessendorf (2001): Phillips or JONSWAP spectrum, time-evolved phasors, `numpy.fft.ifft2`, choppy displacement, simple Blinn-Phong + sky shading, `imageio-ffmpeg` for MP4. Already implemented as a demo in `scripts/wave_synthesis_figures.py`. **Knobs:** U_{10},

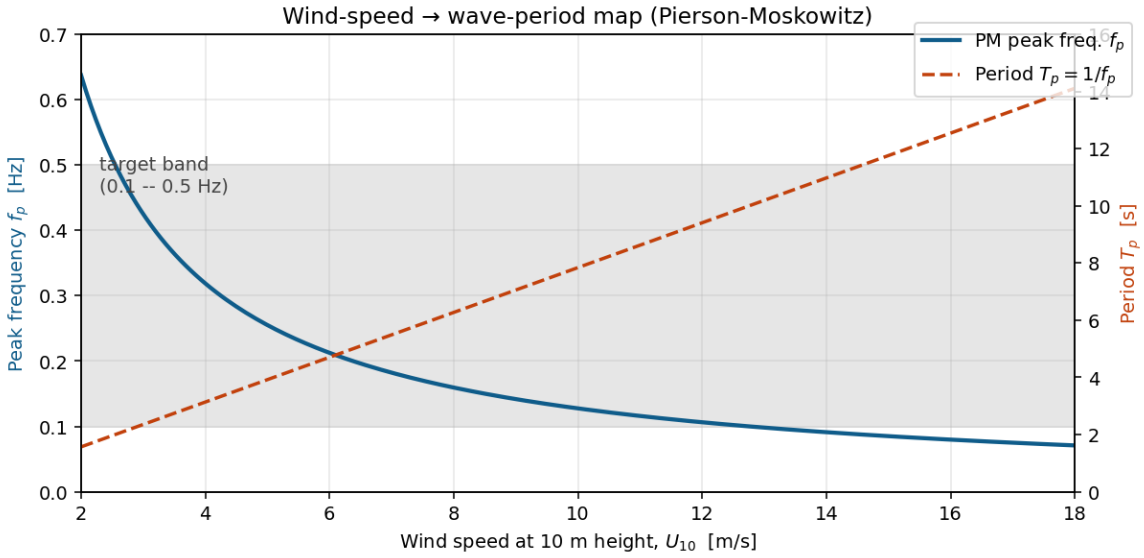

Figure 3: Pierson–Moskowitz peak frequency f_p and corresponding period $T_p = 1/f_p$ as a function of wind speed U_{10}. The shaded region marks the neurophysiological target band (0.1–0.5 Hz). For $f_p = 0.25$ Hz: $U_{10} \approx 5.1$ m/s. For $f_p = 0.10$ Hz (stormy swell): $U_{10} \approx 12.7$ m/s.

target f_p (Hz)	target period T_p (s)	U_{10} (m/s, PM)
0.10 (slow swell)	10.0	12.7
0.15	6.7	8.5
0.20	5.0	6.4
0.25 (mid)	4.0	5.1
0.33	3.0	3.8
0.50 (small chop)	2.0	2.5

Table 1: Wind-speed inputs that hit each entrainment-target frequency under a fully developed Pierson–Moskowitz sea.

wind direction, fetch, depth h, choppiness, foam threshold, sun, camera, seed. **Cost:** seconds to a few minutes for 30 fps×120 s ×1080p on a CUDA GPU. **Realism:** stylised but spectrally correct.

Use for: pilot studies, parameter sweeps, dose-response curves on f_p. The user changes a single argument (U_{10}) to walk through Table 1.

4.2 Tier 2 – Taichi or moderngl shader (interactive)

Same spectral inputs as Tier 1, but rendered in real-time with a Taichi (Hu *et al.*, 2019) kernel or a moderngl raymarched shader (Inigo Quilez's *Seascape* ported as a fragment shader). Render off-screen to a PNG sequence, `ffmpeg` to MP4. **Use for:** interactive parameter tuning before locking the final stimulus values.

4.3 Tier 3 – Blender bpy + Cycles (near photoreal)

Drive Blender's *Ocean* modifier from a Python script:

```
import bpy

```

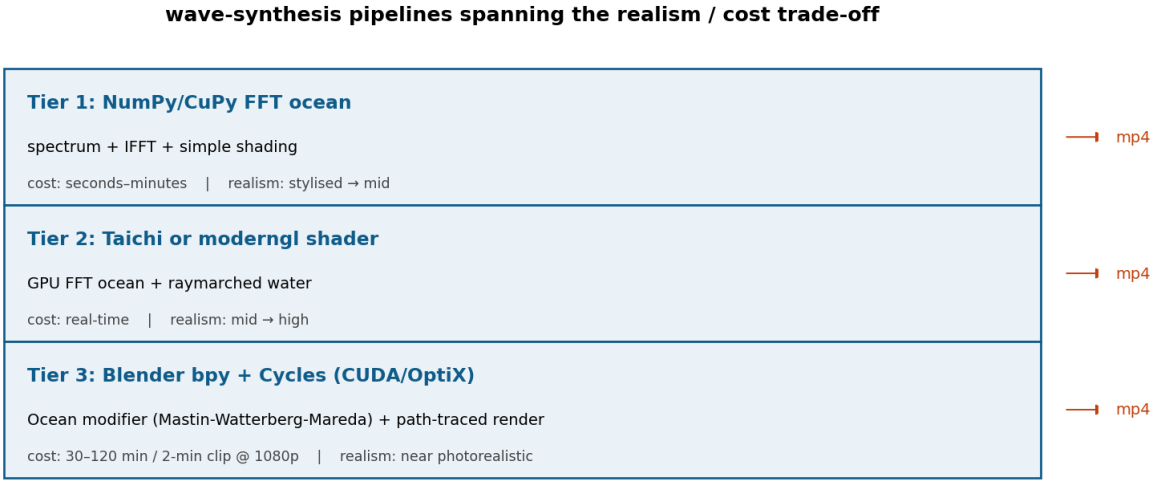

Figure 4: Three-tier wave-synthesis pipeline spanning the realism / cost trade-off. The same numerical knobs flow through all three; only the rendering layer changes.

```
bpy.ops.mesh.primitive_plane_add(size=200)
plane = bpy.context.object
mod = plane.modifiers.new(name="Ocean", type='OCEAN')
mod.spatial_size   = 256
mod.resolution     = 32
mod.wind_velocity  = 5.1            # -> f_p ~ 0.25 Hz
mod.depth          = 200
mod.choppiness     = 1.0
mod.foam_coverage  = 0.4
mod.random_seed    = 42
mod.use_foam       = True

# Animate
scene = bpy.context.scene
scene.frame_start, scene.frame_end = 1, int(120 * 30)
scene.render.fps   = 30
scene.render.engine = 'CYCLES'
scene.cycles.device = 'GPU'

# Time keyframe
mod.time = 0
mod.keyframe_insert(data_path="time", frame=1)
mod.time = 120.0
mod.keyframe_insert(data_path="time", frame=scene.frame_end)

bpy.ops.render.render(animation=True)
```

Sky HDRI, sun position, camera, foam shader, and depth of field are all scriptable. **Cost:** 30–120 min for a 2-min 1080p clip on an RTX-class GPU at moderate samples. **Realism:** near photorealistic. **Use for:** the final experimental stimuli where naturalistic appearance is itself a study variable.

4.4 (Optional) Tier 4 – Mitsuba 3 path-traced

NumPy/Taichi FFT geometry exported as a displacement map, rendered with `mitsuba 3`'s ocean BSDF + sky model. Use only if Cycles isn't realistic enough for a particular shot. BSD-3, fully Python.

5 Validation step (mandatory)

Whichever tier we use, after rendering each clip we must verify that the delivered dominant frequency matches the requested one. The `HNA.modules.video` pipeline is exactly the right tool: run

```
from HNA.modules.video import quantify_video
feats = quantify_video("synth/clip.mp4",
                       include_pixel_spectrum=True)
assert abs(feats.timestack.dominant_freq_hz - f_p_target) < 0.02
```

and confirm both the timestack peak and the modal f_1 land within tolerance of the target f_p. The per-pixel spectrum's `coherence_index` should be high (> 0.9 for a clean spectral ocean). This validation step is doubly important if any AI rendering pass is ever folded on top: that pass can shift phase, alias periods, or introduce side-bands.

6 What we will skip

- **Pure video-diffusion stimuli.** No reliable f_p control; no reproducibility from a numerical seed alone. Useful only as a cosmetic finishing pass.
- **Full 3-D Navier–Stokes / SPH / MPM.** Gives nothing the spectral method doesn't already give for open-water; cost is 1000× higher.
- **Houdini, Embergen.** Commercial; not portable.

7 Proposed module skeleton (next step)

The next deliverable is a module `HNA.modules.wave_synth` structured as:

```
HNA.modules.wave_synth
+- spectra.py          # Phillips, PM, JONSWAP, dispersion
+- fft_ocean.py        # Tessendorf heightfield, choppy + foam
+- render_simple.py    # NumPy Blinn-Phong / sky shading -> mp4
+- render_taichi.py    # Taichi GPU shader (Tier 2)
+- render_blender.py   # bpy script template (Tier 3)
+- validate.py         # round-trip via HNA.modules.video
+- presets.py          # named (calm, swell, storm) parameter packs
```

with a single high-level entry point:

```
from HNA.modules.wave_synth import synthesise

synthesise(out_mp4="stimuli/swell_025Hz.mp4",
           f_p_hz=0.25,                    # -> resolves U10 = 5.1 m/s
           duration_s=120, fps=30, resolution=(1920, 1080),
           tier="numpy",                   # or "taichi", "blender"
           wind_dir_deg=20, fetch_km=80, depth_m=200,
           choppiness=1.0, foam_coverage=0.4, sun_alt_deg=30,
           seed=42)
```

The `tier` argument selects the rendering backend; the spectral parameters are identical across tiers, ensuring that the same f_p can be hit at three realism levels with one call.

References

THUDM CogVideo team. CogVideoX: Text-to-Video diffusion with an Expert Transformer. GitHub: https://github.com/THUDM/CogVideo, 2024.

M. Finch. Effective water simulation from physical models. In *GPU Gems*, ch. 1. Addison-Wesley, 2004.

A. Fournier, W. T. Reeves. A simple model of ocean waves. In *SIGGRAPH '86*, pp. 75–84, 1986.

Genesis-Embodied-AI. Genesis: a generative & universal physics engine. GitHub: https://github.com/Genesis-Embodied-AI/Genesis, 2024.

K. Hasselmann *et al.* Measurements of wind-wave growth and swell decay during the Joint North Sea Wave Project (JONSWAP). *Deutsche Hydrographische Zeitschrift*, A8(12), 1973.

P. Holzschuh *et al.* PhiFlow: a differentiable PDE simulation framework. GitHub: https://github.com/tum-pbs/PhiFlow.

A. Holynski, B. Curless, S. Seitz, R. Szeliski. Animating pictures with Eulerian motion fields. In *CVPR*, 2021.

C. J. Horvath. Empirical directional wave spectra for computer graphics. In *Symposium on Digital Production*, 2015.

Y. Hu *et al.* Taichi: a language for high-performance computation on spatially sparse data structures. *ACM TOG (SIGGRAPH Asia)*, 38(6):201, 2019.

W. Jakob *et al.* Mitsuba 3 renderer. https://www.mitsuba-renderer.org/.

M. Macklin, M. Müller. Position based fluids. *ACM Trans. Graph.*, 32(4):104, 2013.

G. A. Mastin, P. A. Watterberg, J. F. Mareda. Fourier synthesis of ocean scenes. *IEEE Computer Graphics & Applications*, 7(3):16–23, 1987.

W. J. Pierson, L. Moskowitz. A proposed spectral form for fully developed wind seas based on the similarity theory of S. A. Kitaigorodskii. *J. Geophys. Res.*, 69(24):5181–5190, 1964.

J. Stam. Stable fluids. In *SIGGRAPH '99*, pp. 121–128, 1999.

J. Tessendorf. Simulating ocean water. SIGGRAPH course notes, 2001 (revised 2004).

N. Wadhwa, M. Rubinstein, F. Durand, W. T. Freeman. Phase-based video motion processing. *ACM Trans. Graph.*, 32(4):80, 2013.

H.-Y. Wu, M. Rubinstein, E. Shih, J. Guttag, F. Durand, W. T. Freeman. Eulerian video magnification for revealing subtle changes in the world. *ACM Trans. Graph.*, 31(4):65, 2012.